

IF2210 OOP

Java

[Java Interface](#)

[Multiple Inheritance](#)

[Definisi Interface](#)

[Perbedaan Kelas Abstrak dan Interface](#)

[When to use abstract class or interface?](#)

[Observer Pattern](#)

[Java Generics](#)

[Konvensi Penamaan Tipe](#)

[Generic Methods & Constructors](#)

[Subtyping](#)

[Wildcard](#)

[When to use wildcard \(lower/upper bounded\)?](#)

[Java Collection Framework](#)

[Collection Interface](#)

[Set Interface](#)

[List interface](#)

[Queue Interface](#)

[Deque interface](#)

[Map Interface](#)

[Stream API](#)

[Java Exception](#)

[Exception](#)

[Try block](#)

[Catch block](#)

[Finally block](#)

[Try-with-resources](#)

[How to throw exception](#)

[Chained Exception](#)

[Kelas Throwable dan Turunannya](#)

[Java Assertion](#)

[Assertion](#)

[Assertion Statement](#)

[Pengaktifan/Pemeriksaan Asersi](#)

[When to use Assertion?](#)

[Internal Invariant](#)

[Control-Flow Invariant](#)

[Preconditions](#)

[Post conditions](#)

[Class Invariant](#)

[Bad Use of Assertion](#)

[Assertions as Defensive Programming](#)

[Java Multithreading](#)

[Definition](#)

[Process & Thread](#)

[When to use thread?](#)

[Mengimplementasikan Thread](#)

[Thread vs Runnable](#)

[Menjalankan Thread](#)

[Thread.sleep\(int\)](#)

[Memakai Interface Runnable](#)

[Thread dengan Anonymous Class](#)

[Thread dengan Lambda Expression](#)

[Timer](#)

[Menjadwalkan Pekerjaan](#)

[Siklus Hidup Thread](#)

[Thread Priority/Scheduling](#)

[Masalah Sinkronisasi](#)

[Wait dan Notify](#)

[wait, notify, dan notifyAll](#)

[Java Reflection](#)

[Implementasi bytecode pada Java](#)

[Class Loader](#)

[Reflection API](#)

[Kelas Class](#)

[Kelas Field dan kelas Method](#)

[Use Case Reflection](#)

Java Interface

Multiple Inheritance

- Problem: Java hanya mendukung single inheritance.
- Solution: untuk memfasilitasi multiple inheritance, kita dapat menggunakan **interface**

Definisi Interface

- Sebuah interface seperti halnya class kecuali:
 - semua instance methods harus berupa abstract methods
 - hanya method interfaces yang dideklarasikan
- Implementasi method harus disediakan oleh setiap class non-abstract yang mengimplementasikan interface.
- Interface tidak bisa berisi constructor methods

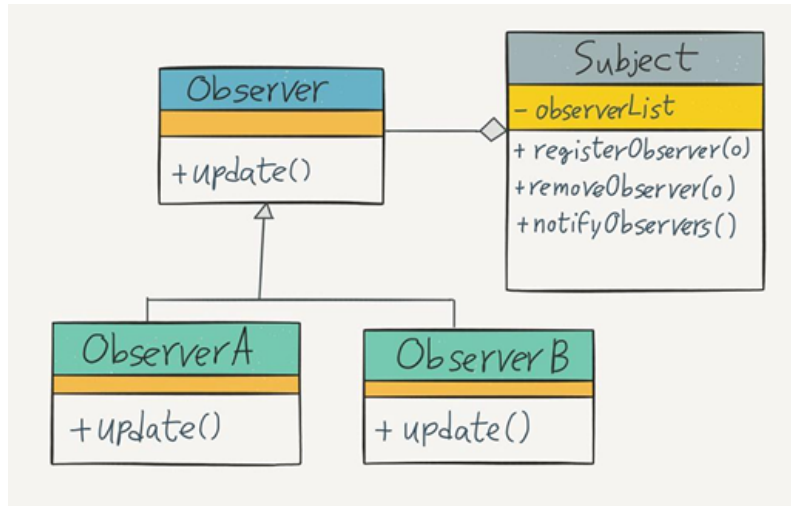
Perbedaan Kelas Abstrak dan Interface

- Kelas abstrak boleh memiliki method yang sudah diimplementasikan, sedangkan interface harus “kosong” (tidak ada method yang diimplementasikan pada interface karena implementasi method interface dilakukan oleh kelas yang implements interface tersebut)
- Kelas hanya boleh diturunkan dari satu kelas, namun boleh mengimplementasikan banyak interface.

When to use abstract class or interface?

- Gunakan kelas abstrak jika sudah ada algoritma yang bisa diimplementasikan di kelas tersebut.
 - Kelas abstrak biasanya diturunkan menjadi kelas-kelas yang masih sangat berhubungan satu sama lain, seperti `AbstractList` diturunkan menjadi `LinkedList`, `ArrayList`, dst. Sedangkan interface dapat diimplementasi oleh kelas-kelas yang tidak berhubungan.
- Gunakan interface jika hanya memberi kontrak, misalnya interface `Measurable` untuk menyatakan objek yang bisa diukur keliling dan luasnya.
- Gunakan interface jika ingin mengaplikasikan multiple inheritance.

Observer Pattern



Java Generics

- Konsep generik menambah stabilitas kode dengan membantu deteksi bug saat compile time
- Kelas generik pada java diimplementasikan sebagai parameter tipe
- Hasil kompilasi kode kelas generik tetap hanya satu kelas tetapi dengan parameter tipe yang diganti dengan tipe riil pada saat runtime
- Keuntungan:
 - meningkatkan expressive power
 - meningkatkan type safety
 - mengeksplisitkan parameter tipe dan mengimplisitkan type casting

Konvensi Penamaan Tipe

- E : Element (used extensively by the java collections framework)
- K : Key
- N : Number
- T: Type
- V : Value
- S, U, V, etc. : 2nd, 3rd, 4th types

Generic Methods & Constructors

- Type parameter juga dapat digunakan pada method dan konstruktor menjadi generic methods dan generic constructors

- Compiler dapat mengetahui tipe parameter pada pemanggilan method generik dari tipe argumennya

```
public class Box<T> {
    // ...
    public static <U> void fillBoxes(U u, List<Box<U>> boxes) {
        for (Box<U> box: boxes) box.put(u);
    }
    // ...
}

// Somewhere else...
Crayon red = new Crayon("red");
List<Box<Crayon>> crayonBoxes = new ArrayList<>();

Box.<Crayon>fillBoxes(red, crayonBoxes);
Box.fillBoxes(red, crayonBoxes); // compiler infers that U is Crayon
```

- Kita dapat membatasi tipe-tipe apa saja yang diperbolehkan untuk masuk sebagai parameter dengan menggunakan **bounded type parameters**

```
public class Box<T> {
    // ...
    public <U extends Number> void boundedInspect(U u) {
        System.out.println("T: " + t.getClass().getName());
        System.out.println("U: " + u.getClass().getName());
    }
    // ...
}
```

- Apakah bisa bound dengan interface? Tetap gunakan keyword extends, jika ada lebih dari satu gunakan operator &. Contohnya seperti berikut

```
public <T extends SomeInterface> void a(T t) {...}
public <U extends Number & SomeInterface> void b(U u) {...}
```

Subtyping

- Jika D extends atau implements B, maka semua reference ke B dapat di-assign dengan objek dari kelas D. Jika Integer extends Number, maka kode di bawah ini valid:

```
Number aNumber;
Integer anInt = new Integer(10);
aNumber = anInt; // OK
```

- Hal ini berlaku juga pada pemanggilan method:

```
public void someMethod(Number n) { /*...*/ }

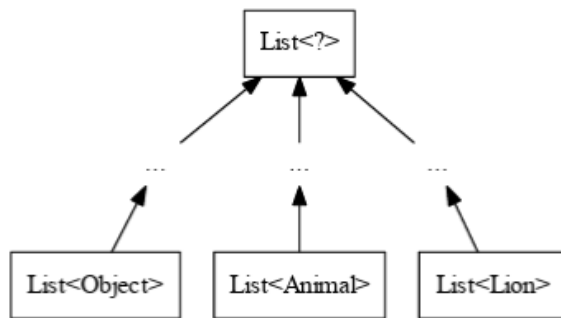
// somewhere else...
someMethod(new Integer(10)); // OK
someMethod(new Double(10.0)); // OK
```

- dan tipe generik:

```
Box<Number> box = new Box<>();
box.put(new Integer(10)); // OK
box.put(new Double(10.0)); // OK
```

Wildcard

- Object adalah leluhur semua kelas di Java, namun untuk generik, contohnya List<Object> bukan leluhur dari semua list. Melainkan, leluhur dari semua list adalah List<?> ("list of unknown") → AKA **wildcard** type



- Upper bounded wildcard: <? extends Something>
- Lower bounded wildcard: <? super Something>

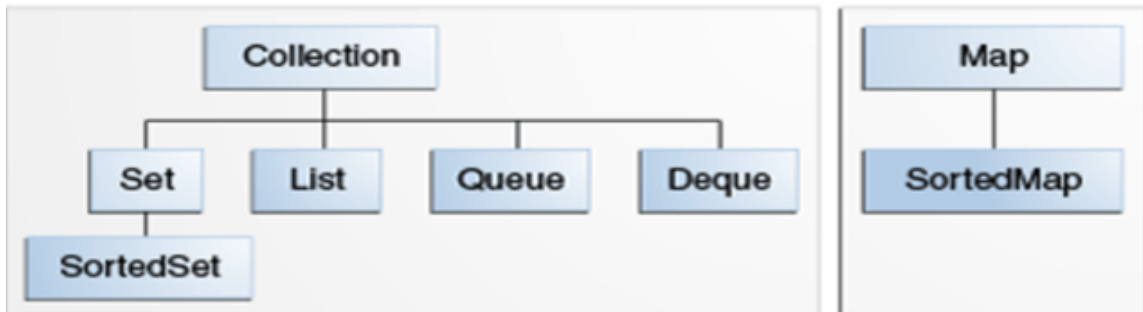
When to use wildcard (lower/upper bounded)?

- Jika variabel hanya dibaca, gunakan upper bound
- Jika variabel hanya untuk modifikasi, gunakan lower bound
- Jika variabel dibaca dan dimodifikasi, jangan gunakan wildcard

Java Collection Framework

- A collections framework is a unified architecture for representing and manipulating collections
- All collections frameworks contain the following;
 - **interfaces:** the abstract data types that represent collections
 - **implementations:** the concrete implementations of the collection interfaces

- **algorithms:** the methods that perform useful computations, such as searching and sorting, on objects that implement collection interfaces
- Core collection interfaces



Collection Interface

- A Collection represents a group of objects known as its elements
- A collection interface declares:
 - methods that perform basic operations, such as:
 - `int size()`,
 - `boolean isEmpty()`,
 - `boolean contains(Object element)`,
 - `boolean add(E element)`,
 - `boolean remove(Object element)`, and
 - `Iterator<E> iterator()`
 - methods that operate on entire collections
 - `boolean containsAll(Collection<?> c)`,
 - `boolean addAll(Collection<? extends E> c)`,
 - `boolean removeAll(Collection<?> c)`,
 - `boolean retainAll(Collection<?> c)`, and
 - `void clear()`
 - additional methods for array operations, such as:
 - `Object[] toArray()`
 - `<T> T[] toArray(T[] a)`

Set Interface

- A Set is a Collection that cannot contain duplicate elements
- Contains only methods inherited from Collection and adds the restriction that duplicate elements are prohibited
- Three general-purpose Set implementations: HashSet, TreeSet, LinkedHashSet

List interface

- A List is a Collection that preserves insertion order
- May contain duplicate
- includes operations for:
 - positional access
 - search
 - iteration
 - range-view
- Two general-purpose list implementations: ArrayList and LinkedList

Queue Interface

- A Queue is a collection for holding elements prior to processing
- Besides basic Collection operations, queues provide additional insertion, removal, and inspection operations

Deque interface

- double-ended queue → a linear collection of elements that supports the insertion and removal of elements at both end points
- Methods provided are to insert, remove, and examine the elements
- predefined classes like ArrayDeque and LinkedList implement the Deque interface

Map Interface

- A Map is an object that maps keys to values
- Cannot contain duplicate keys, each key can map to at most one value
- Includes methods for basic operations (such as put, get, remove, containsKey, containsValue, size, and empty), bulk operations (such as putAll and clear), and collection views (such as keySet, entrySet, and values).
- Three general-purpose Map implementations: HashMap, TreeMap, LinkedHashMap

Stream API

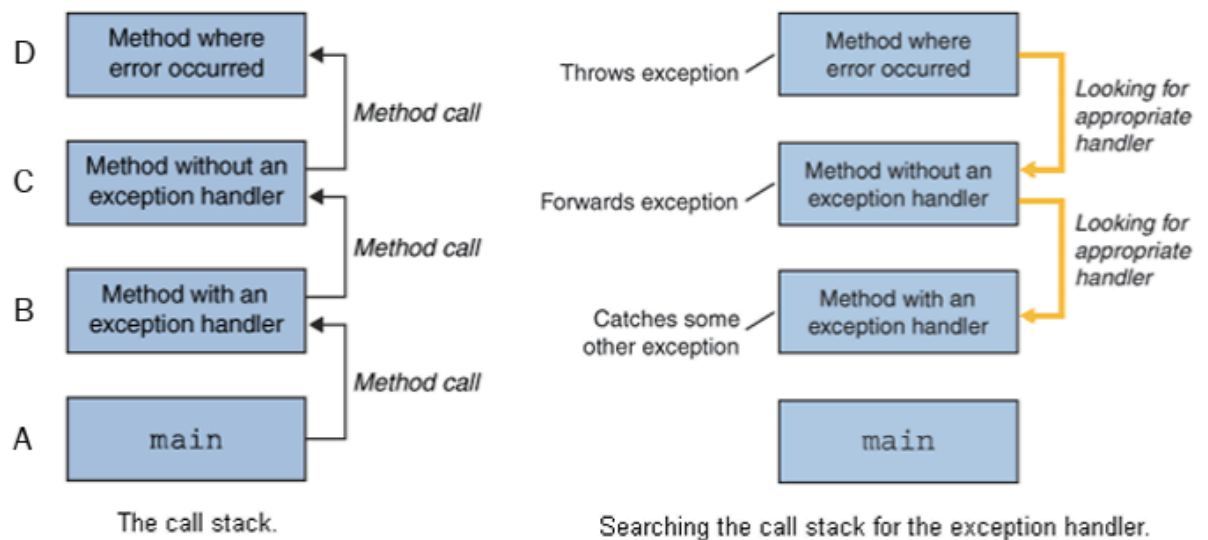
Pada java stream API, secara umum ada 4 jenis proses:

- filter: dari stream menjadi stream bertipe sama dengan jumlah elemen yang bisa lebih sedikit dari stream aslinya
- map: dari stream menjadi stream (bisa bertipe lain) dengan jumlah elemen yang sama dengan stream aslinya
- reduce: dari stream menjadi satu nilai
- forEach: melakukan sesuatu terhadap semua elemen stream

Java Exception

Exception

- Exceptional event adalah sebuah event yang terjadi saat eksekusi program yang mengganggu alur normal dari instruksi program
- Ketika terjadi sebuah error pada sebuah method, maka method tersebut akan **menciptakan sebuah objek** dan **melempar ke runtime system**. Objek yang diciptakan tsb disebut **exception object**.
 - Exception object mengandung informasi tentang error, termasuk tipe dan state program ketika error terjadi
- Setelah exception dilempar, runtime system akan mencari exception handler yang akan meng-catch exception tadi.



Try block

- Berisi kode yang mungkin memunculkan exception
- Try block bisa dibuat untuk setiap kode yang mungkin menimbulkan exception
- Bisa juga dengan mengumpulkan banyak kode dalam sebuah try block

Catch block

- Berisi kode yang merupakan exception handler
- Menangani exception dengan tipe yang sesuai dengan tipe yang ditujukan pada argumen

- Satu try block dapat diikuti dengan lebih dari satu catch block

Finally block

- Berisi kode yang **pasti** dieksekusi setelah try block
- Finally block akan dieksekusi baik terjadi exception atau tidak
- Meskipun dalam try block atau catch block ada return atau throw, finally block akan tetap dieksekusi terlebih dahulu sebelum melanjutkan return/throw

```
try {  
    // ...  
  
} catch(/*...*/) {  
    // ...  
  
} finally {  
  
    if (out != null) {  
        System.out.println("Closing PrintWriter");  
        out.close();  
    } else {  
        System.out.println("PrintWriter not open");  
    }  
  
}
```

Try-with-resources

- Dapat digunakan untuk mendeklarasikan variabel yang akan digunakan dalam try block
- Variabel tersebut akan otomatis di-close() setelah blok selesai tanpa perlu finally block
- variabel harus mengimplementasi **interface AutoCloseable**.

How to throw exception

- Menggunakan throw statement
- Membutuhkan sebuah argumen berupa objek throwable yang merupakan instansiasi dari kelas Throwable atau turunannya
- Jika yang dilempar adalah checked exception, deklarasi method harus disertai keyword throws seperti `void doSomethingTo(Object x) throws SomeException { ... }`

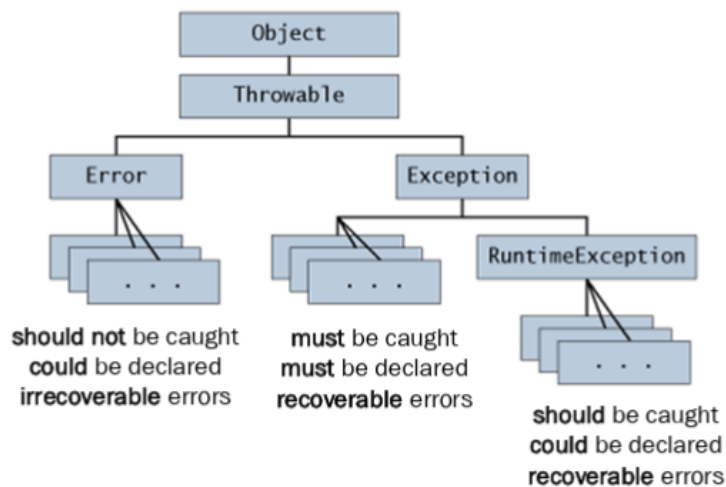
Chained Exception

- Method bisa merespon terjadinya exception dengan melempar exception lagi

- Digunakan untuk membungkus exception dengan informasi tambahan

```
try {
    // ...
} catch (IOException e) {
    throw new SampleException("Other IOException", e);
}
```

Kelas Throwable dan Turunannya



Java Assertion

Assertion

- An assertion is a statement that enables you to test your assumptions about your program
- Each assertion contains a boolean expression that you believe will be true when the assertion executes
 - if it is not true, the system will throw an error
 - by verifying that the boolean expression is indeed true, the assertion confirms your assumptions about the behavior of your program, increasing your confidence that the program is free of errors
- Experience has shown that writing assertions while programming is one of the quickest and most effective ways to detect and correct bugs

Assertion Statement

- The assertion statement has two forms
 - the first, simpler form is: **`assert Expression;`**
where Expression is a boolean expression. When the system runs the assertion, it evaluates Expression and if it is false it will throw an AssertionError with no detail message.

for example, asersi bahwa speed pasti < 300000:
`assert (speed < 300000);`
 - the second form is: **`assert Expression1 : Expression2;`**
where Expression1 is a boolean expression and Expression2 is an expression that has a value (it cannot be an invocation of a method that is declared as a void)

for example, asersi ingin diberi pesan yang informatif:
`assert (speed < 300000) : "kecepatan melebihi kecepatan cahaya";`

Pengaktifan/Pemeriksaan Asersi

- By default, asersi tidak akan diperiksa oleh JVM
- Asersi diperiksa jika program dijalankan dengan option -ea (enable assertion); i.e.
`java -ea Speed`

When to use Assertion?

- Gunakan assertion pada situasi:
 - internal invariants
 - control-flow invariants
 - preconditions, postconditions, dan class invariants
- Jangan gunakan untuk:
 - cek argumen method yang bersifat publik → karena argumen tsb merupakan bagian dari kontrak
 - melakukan aksi/operasi yang benar yang dilakukan oleh program → karena assertion bisa di-disable

Internal Invariant

- › In the absence of an assertion facility, many programmers use comments in the following way:

```
if (i%3 == 0) {  
    ...  
} else if (i%3 == 1) {  
    ...  
} else { /* i%3 == 2 */  
    ...  
}
```

- › When your code contains a construct that asserts an invariant, you should change it to an assert. To change the above example (where an assert protects the else clause in a multiway if-statement), you might do the following:

```
if (i % 3 == 0) {  
    ...  
} else if (i%3 == 1) {  
    ...  
} else {  
    assert i%3 == 2;  
    ...  
}
```

Control-Flow Invariant

- › Another good candidate for an assertion is a switch statement with no default case.

```
switch(suit) {  
    case Suit.CLUBS:  
        ...  
        break;  
    case Suit.DIAMONDS:  
        ...  
        break;  
    case Suit.HEARTS:  
        ...  
        break;  
    case Suit.SPADES:  
        ...  
}
```

- › The programmer probably assumes that one of the four cases in the above switch statement will always be executed. To test this assumption, add the following default case:

```
switch(suit) {  
    case Suit.CLUBS:  
        ...  
        break;  
    case ...  
        ...  
        break;  
    default:  
        assert false;  
}
```

Preconditions

- Kontrak bahwa pemakai fungsi/prosedur harus menaati suatu prekondisi/initial state yang ditetapkan ketika mendefinisikan fungsi/prosedur
- Selama ini hanya ditulis sebagai komentar, namun dapat di-enforce secara explicit dengan pemeriksaan kasus (if ... then throw ...) atau sebagai asersi
- Bdsk konvensi, prekondisi pada method public di-enforce dengan mengecek secara eksplisit dalam method untuk memperoleh suatu exception, contohnya:

```
/**
 * Sets the refresh rate.
 *
 * @param rate refresh rate, in frames per second.
 * @throws IllegalArgumentException if rate <= 0 or
 *         rate > MAX_REFRESH_RATE.
 */
public void setRefreshRate(int rate) {
    // Enforce specified precondition in public method
    if (rate <= 0 || rate > MAX_REFRESH_RATE)
        throw new IllegalArgumentException("Illegal rate: " + rate);
    setRefreshInterval(1000/rate);
}
```

- Namun, jika ada prekondisi pada method yang **tidak public** dan author dari kelas tersebut percaya bahwa prekondisi harus berlaku tanpa mempedulikan perilaku user/client thd kelas, maka asersi cocok untuk digunakan. Contohnya:

```
/**
 * Sets the refresh interval (must correspond to a legal frame rate).
 * @param interval refresh interval in milliseconds.
 */
private void setRefreshInterval(int interval) {
    // Confirm adherence to precondition in nonpublic method
    assert interval > 0 && interval <= 1000/MAX_REFRESH_RATE;

    ... // Set the refresh interval
}
```

Post conditions

- Kontrak bahwa penyedia fungsi/prosedur harus menaati post condition/final state yang ditetapkan saat mendefinisikan fungsi/prosedur
- Selama ini hanya dituliskan sebagai komentar, sekarang sebaiknya dituliskan sebagai asersi.
- Postconditions checks are best implemented via assertions whether they are specified in public methods or not.

- Contohnya:

```
/**
 * Returns a BigInteger whose value is (this-1 mod m).
 * @param m the modulus.
 * @return this-1 mod m.
 * @throws ArithmeticException m <= 0, or this BigInteger
 *         has no multiplicative inverse mod m (that is, this BigInteger
 *         is not relatively prime to m).
 */
public BigInteger modInverse(BigInteger m) {
    if (m.signum <= 0)
        throw new ArithmeticException("Modulus not positive: " + m);
    if (!this.gcd(m).equals(ONE))
        throw new ArithmeticException(this + " not invertible mod " + m);

    ... // Do the computation

    assert this.multiply(result).mod(m).equals(ONE);
    return result;
}
```

Class Invariant

- A class invariant is a type of internal invariant that applies to every instance of a class at all times, except when an instance is in transition from one consistent state to another.
- A class invariant can specify the relationships among multiple attributes, and should be true before and after any method completes
- For example, suppose you implement a balanced tree data structure of some sort. A class invariant might be that the tree is balanced and properly ordered.

Bad Use of Assertion

- Argument checking in public methods

- To Do any work that your app requires for correct operation

- › Because assertions may be disabled, programs must not assume that the boolean expression contained in an assertion will be evaluated. Violating this rule has dire consequences. For example, suppose you wanted to remove all of the null elements from a list names, and knew that the list contained one or more nulls. It would be wrong to do this:

```
// Broken! - action is contained in assertion
assert names.remove(null);
```

- › The program would work fine when asserts were enabled, but would fail when they were disabled, as it would no longer remove the null elements from the list. The correct idiom is to perform the action before the assertion and then assert that the action succeeded:

```
// Fixed - action precedes assertion
boolean nullsRemoved = names.remove(null);
assert nullsRemoved;
// remove() runs whether or not asserts are enabled
```

- › As a rule, **the expressions contained in assertions should be free of side effects**: evaluating the expression should not affect any state that is visible after the evaluation is complete. One exception to this rule is that assertions can modify state that is used only from within other assertions. An idiom that makes use of this exception is presented later in this document.

Assertions as Defensive Programming

- assertions don't allow a programmer to control response to unexpected conditions (should use exception handling instead)
- ensure that **execution halts** whenever an expected condition is not met
- typically used to **check inputs** to functions, but can be used anywhere
- can be used to **check outputs** of a function to avoid propagating bad values
- can make locating a source of a bug easier

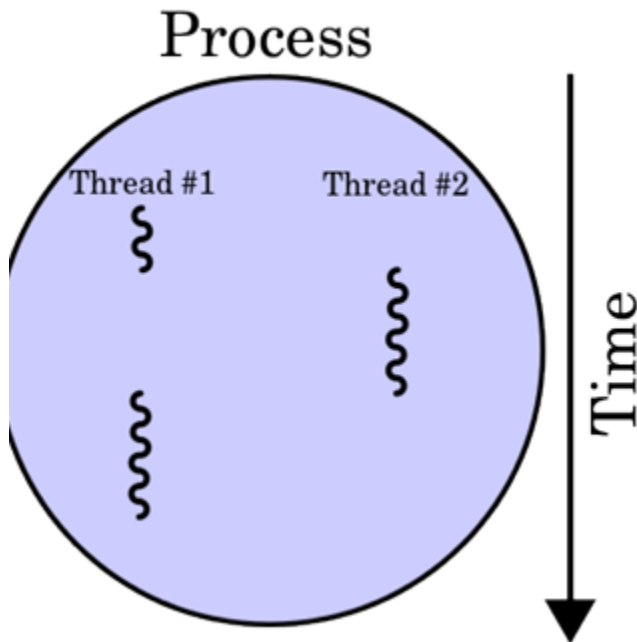
Java Multithreading

Definition

- Multithreading is a type of execution model that allows **multiple threads to exist** within **the context of a process** such that they execute **independently** but **share their process resources**

Process & Thread

- A thread of execution is the smallest sequence of programmed instructions that can be managed independently by a scheduler



When to use thread?

- Pada aplikasi GUI
- aplikasi yang menangani banyak event dalam satu waktu
- aplikasi asinkron

Mengimplementasikan Thread

- Setiap thread berisi kode apa yang ingin dijalankan
- Ada dua cara membuat thread:
 - menurunkan dari kelas *Thread*
 - Override method *run()*, defaultnya tidak melakukan apa-apa
 - mengimplementasikan interface *Runnable*
 - mengimplementasikan method *run()*

Thread vs Runnable

- Thread memiliki method yang berhubungan dengan Thread, sehingga untuk setiap interface Runnable tetap butuh objek thread

```
class MyThread extends Thread {
    String s;

    public MyThread(String str) {
        s = str;
    }

    @Override
    public void run() {
        while (true) {
            System.out.println(s);
            try { Thread.sleep(1000); }
            catch (Exception e) {}
        }
    }
}
```

Menjalankan Thread

```
public class MultithreadingDemo1 {
    public static void main(String[] args) {

        new MyThread("Jamaica").start();
        new MyThread("Fiji").start();
    }
}
```

- Output: akan muncul "Jamaica" dan "Fiji" bergantian.

Thread.sleep(int)

- Menunda eksekusi selama n milisekon
- Delay dalam kasus ini agar output lebih terlihat

Memakai Interface Runnable

```
class MyRunnable implements Runnable {
    String s;

    public MyRunnable(String str) {
        s = str;
    }

    @Override
    public void run() {
        while (true) {
            System.out.println(s);
            try { Thread.sleep(1000); }
            catch (Exception e) {}
        }
    }
}

public class MultithreadingDemo2 {
    public static void main(String[] args) {
        Runnable r1, r2;
        r1 = new MyRunnable("Jamaica");
        r2 = new MyRunnable("Fiji");

        new Thread(r1).start();
        new Thread(r2).start();
    }
}
```

- Perhatikan bahwa objek Thread tetap perlu diciptakan, dengan objek Runnable sebagai argumen konstruktornya

Thread dengan Anonymous Class

```
public class MultithreadingDemo3 {
    public static void main(String[] args) {
        Runnable r = new Runnable() {
            @Override
            public void run() {
                while (true) {
                    System.out.println("Jamaica");
                    try { Thread.sleep(1000); }
                    catch (Exception e) {}
                }
            }
        };
        new Thread(r).start();
    }
}
```

Thread dengan Lambda Expression

```
public class MultithreadingDemo4 {
    public static void main(String[] args) {
        Runnable r = () -> {
            while (true) {
                System.out.println("Jamaica");
                try { Thread.sleep(1000); }
                catch (Exception e) {}
            }
        };
        new Thread(r).start();
    }
}
```

Timer

- Selain Thread, Java menyediakan Timer
- Timer dapat digunakan untuk **menjadwalkan** pekerjaan
 - tidak perlu membuat thread terpisah secara manual
 - menyederhanakan kasus tertentu
- Hal yang bisa dilakukan oleh Timer bisa dilakukan oleh Thread

Menjadwalkan Pekerjaan

- Menggunakan kelas TimerTask dan Timer di java.util
- Membuat kelas turunan TimerTask

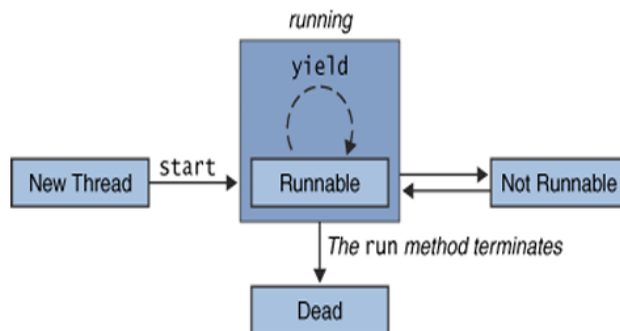
```
import java.util.Timer;
import java.util.TimerTask;

class MyTask extends TimerTask { // berisi pekerjaan yg dilakukan
    private String s;
    public MyTask(String str) { s = str; }
    @Override
    public void run() {
        System.out.println(s);
    }
}

public class TimerDemo {
    public static void main(String[] args) {
        Timer timer = new Timer();
        long delay = 1000, period = 2000; // in milliseconds
        timer.schedule(new MyTask("Once"), delay);
        timer.scheduleAtFixedRate(new MyTask("Repeated"), delay, period);
    }
}
```

Siklus Hidup Thread

- Thread hidup selama method run() masih berjalan



- Thread bisa berhenti sementara/di-suspend karena 3 hal:

- menunggu I/O
- method sleep dipanggil untuk Thread tersebut
- menunggu event karena pemanggilan wait

Thread Priority/Scheduling

- Thread bisa diberi prioritas berbeda
 - dari konstruktor
 - dengan method setPriority()
- Thread yang lebih tinggi prioritasnya lebih sering berjalan

Masalah Sinkronisasi

- Dua hal yang berjalan bersamaan akan sangat mungkin bertabrakan, contohnya ketika akses variabel bersamaan atau write bersamaan ke file yang sama
- Agar satu method selesai melakukan pekerjaannya sebelum diinterupsi, kita dapat menambahkan keyword **synchronized**: *public synchronized void write(){}*
- Ketika suatu method synchronized di suatu objek dipanggil, objek itu akan dikunci agar tidak ada yang bisa mengakses method dalam objek tersebut yang sifatnya synchronized
- Jika method tidak di-synchronize, gunakan:

```
synchronized (object) {
    // Statement yang akan dilakukan
    // dalam keadaan objek ter-lock
}
```

- - Perhatikan bahwa keduanya ini sama:

```
class Whatever {
    synchronized ReturnType method(...) {
        /*BODY*/
    }
}

class Whatever {
    ReturnType method(...) {
        synchronized (this) {
            /*BODY*/
        }
    }
}
```

Wait dan Notify

- Dalam permasalahan tertentu, terkadang method harus menunggu agar method lain selesai bekerja
- Kasus klasik adalah di design pattern Producer-Consumer, di mana ada sebuah producer yang menghasilkan sesuatu dan sebuah consumer yang menghabiskan produk producer.
- Contoh kasus:
 - Producer Pizza.
 - Consumer Pizza.
 - Objek PizzaBox (untuk meletakkan Pizza).
 - PizzaBox hanya bisa diisi satu pizza.
 - Jika Pizza belum diambil, Producer tidak boleh menaruh Pizza lagi.
 - Jika Pizza tidak ada, consumer harus menunggu Pizza.
- Contoh implementasi bdsk kasus:

```
class PizzaBox {
    Pizza current;

    // mengambil pizza
    synchronized Pizza get() {
        return current;
    }

    // menaruh pizza
    synchronized void put(Pizza p) {
        current = p;
    }
}

class PizzaProducer extends Thread {
    PizzaBox pizzaBox;

    PizzaProducer(PizzaBox pb) {
        pizzaBox = pb;
    }

    @Override
    public void run() {
        while (true) {
            Pizza p = new Pizza(); // buat pizza

            // pizza sudah jadi, taruh di box
            pizzaBox.put(p);
        }
    }
}

class PizzaConsumer extends Thread {
    PizzaBox pizzaBox;

    PizzaConsumer(PizzaBox pb) {
        pizzaBox = pb;
    }

    @Override
    public void run() {
        while (true) {
            Pizza p = pizzaBox.get(); //

            // makan pizza ...
        }
    }
}

public class PizzaDemo {
    public static void main(String[] args) {
        PizzaBox box = new PizzaBox();
        new PizzaProducer(box).start();
        new PizzaConsumer(box).start();
    }
}
```

- Masalah pada implementasi belum menangani kemungkinan
 - ada pizza tapi producer mencoba menaruh pizza lagi: pizza yang lama tertimpa

- tidak ada pizza tetapi consumer berusaha memakan pizza: consumer mendapatkan pizza yang lama
- Sehingga harus ada variabel yang mencatat saat ini sedang ada pizza atau tidak
- Solusi: ketika menunggu pizza.get() harus menunggu sampai di-notify bahwa pizza sudah datang dengan method wait(). ketika menaruh pizza.put() harus diberitahu bahwa pizza sudah diambil dengan method notify()

```
class PizzaBox {
    Pizza current;
    boolean available = false;

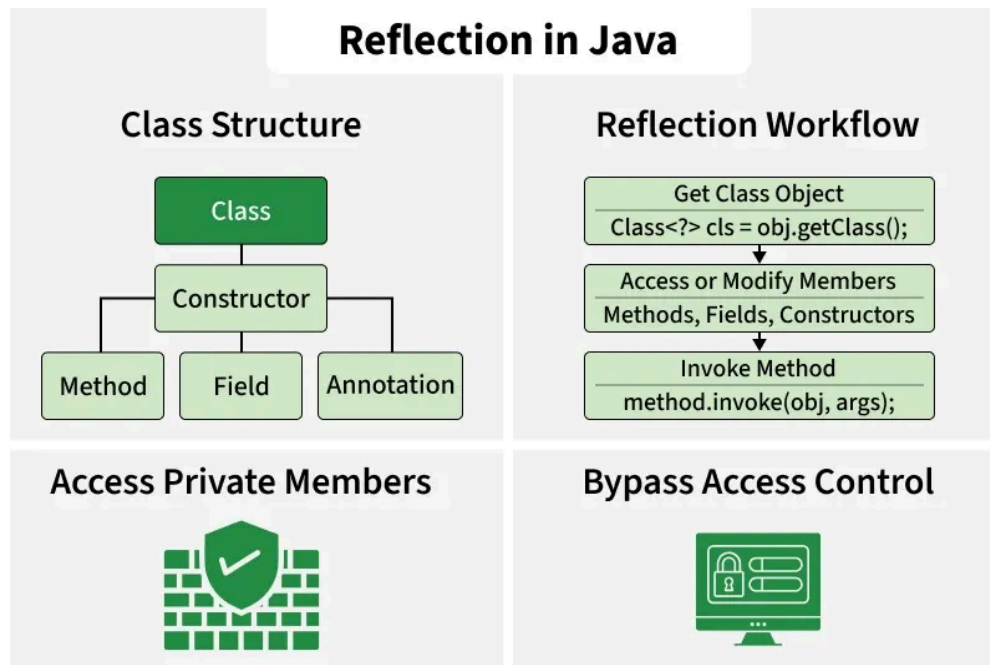
    synchronized Pizza get() {
        while (!available) {
            try { wait(); } // tunggu producer
            catch (InterruptedException e) {}
        }
        available = false; // sudah diambil
        notify(); // beritahu bahwa sudah diambil
        return current;
    }

    synchronized void put(Pizza p) {
        while (available) {
            try { wait(); } // tunggu sampai diambil
            catch (InterruptedException e) {}
        }
        current = p;
        available = true; // pizza siap diambil
        notify(); // beritahu pizza sudah siap
    }
}
```

wait, notify, dan notifyAll

- wait
 - menunggu sampai ada notify() atau notifyAll()
- notify
 - memberi sinyal agar wait berhenti
 - hanya memberitahu satu objek
- notifyAll
 - memberitahu semua objek yang menunggu

Java Reflection



malas nulis klo ga jelas baca dari sini

<https://www.geeksforgeeks.org/java/reflection-in-java/>

Implementasi bytecode pada Java

- semua program java akan dikompilasi menjadi bytecode sebelum eksekusi
 - bytecode adalah bahasa mesin untuk JVM yang umumnya disimpan dalam bentuk file .class
- bytecode yang dieksekusi tidak wajib ada dalam file

Class Loader

- bagian dari JVM yang me-load kelas ke dalam memori
- by default class loader bawaan Java, kelas dapat dimuat dari filesystem/file .jar/ jaringan (i.e. dalam kasus applet).
- kita dapat membuat custom class loader dengan menurunkan kelas abstrak **java.lang.ClassLoader**

Reflection API

- **problem:** class loader menciptakan kelas baru saat runtime namun kelas tsb mungkin tidak bisa digunakan karena programmer tidak tahu apa saja method dan property pada kelas tsb

- **solusi:** digunakan reflection API untuk bisa melihat dan mengeksekusi isi sebuah kelas baik yang dikenal maupun tak dikenal saat kompilasi

Kelas Class

- untuk mengakses isi sebuah kelas, nama lengkapnya adalah `java.lang.Class`
- kelas `Class` harus diinstansiasi dengan cara khusus (bukan dengan `new`)
 - sebuah objek dari kelas `Class` berkorespondensi dengan sebuah nama kelas yang sudah dikenal Java
 - Instansiasi dapat dilakukan dengan 3 cara:
 - nama kelas
 - literal nama kelas
 - objek yg sudah dimiliki sebelumnya
- kelas `Class` memiliki banyak method, di antaranya yang penting adalah:
 - `newInstance()` untuk membuat objek baru
 - `getClasses()` untuk mengembalikan interface dan kelas parent
 - `getFields()` dan `getMethods()` untuk mengembalikan daftar field (atribut) dan method yang dimiliki.
 - jika nama method/field sudah diketahui, ada juga method `getField()` dan `getMethod()` tunggal
 - Instansiasi objek dengan objek `Class` dapat dilakukan dengan:


```
Class c = String.class;
String s = (String)c.newInstance();

Class cv = class.forName("java.lang.Vector");
Vector v = (Vector)cv.newInstance();
```

Kelas Field dan kelas Method

- Kelas `Field` memungkinkan akses terhadap field/atribut dalam sebuah kelas
 - mendapatkan nama, tipe, membaca, dan mengeset nilai field
- kelas `Method` memungkinkan akses terhadap sebuah method dalam sebuah kelas
 - mendapatkan nama, return value, dan parameter sebuah method
 - memanggil method

Use Case Reflection

- digunakan IDE untuk design GUI
- digunakan application server
- digunakan untuk plugin

- plugin adalah kelas/komponen yang bisa ditambahkan tanpa perlu rekompilasi aplikasi
 - plugin diimplementasikan sebagai kelas: untuk mempermudah, plugin pasti mengimplementasikan interface FileLoader
 - plugin diletakkan di sebuah direktori tertentu

Design Pattern

<https://refactoring.guru/design-patterns>

Pattern ≠ Framework

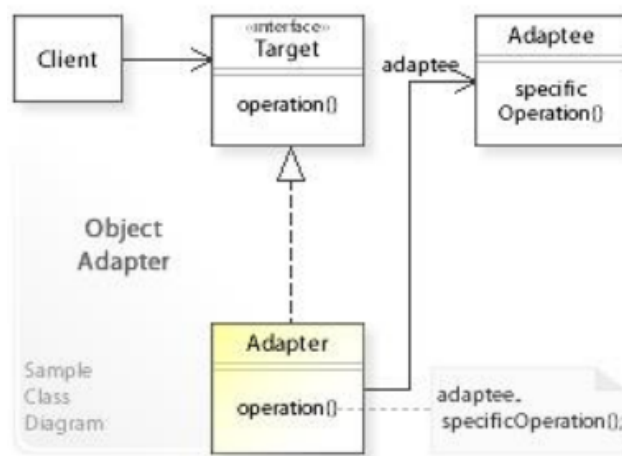
- Pattern: alternative ways of modeling a situation
- Framework: choosing a particular model

Gang of Four Book

- Berisi 23 design pattern yang dikelompokkan dalam 3 kategori:
 - Creational: mengenai penciptaan objek
 - Abstract factory
 - Builder
 - Factory method
 - Prototype
 - Singleton
 - Structural: mengenai komposisi objek
 - Adapter
 - Bridge
 - Composite
 - Decorator
 - Facade
 - Flyweight
 - Proxy
 - Behavioral: mengenai komunikasi antar objek
 - Chain of responsibility
 - Command
 - Interpreter
 - Iterator
 - Mediator
 - Memento
 - Observer
 - State

- Strategy
- Template method
- Visitor
- Setiap pattern dalam buku memiliki setidaknya 4 elemen:
 - Pattern name,
 - Problem (when to apply the pattern)
 - Solution (elements that make up the design)
 - Consequences (results, trade-offs)

Adapter Pattern



- AKA wrapper
- **Intent:** convert the interface of a class into another interface clients expect. Adapter lets classes work together that couldn't otherwise because of incompatible interfaces
- Use when:
 - you want to use an existing class, and its interface doesn't match the one you need
 - you want to create a reusable class that cooperates with unrelated or unforeseen classes, that is, classes that don't necessarily have compatible interfaces
 - (object adapter only) you need to use several existing subclasses, but it's impractical to adapt their interfaces by subclassing everyone. An object adapter can adapt the interface of its parent class
- Example:

Ponsel android memiliki interface micro USB untuk ngecas, sedangkan iPhone menggunakan lighting. iPhone bisa saja dicas dengan kabel micro USB selama ada konektor atau adapter yang membuat iPhone seolah memiliki interface micro USB.

```

class Android implements MicroUsbPhone {
    private boolean connector;

    @Override
    public void useMicroUsb() {
        connector = true;
        System.out.println("MicroUsb connected");
    }

    @Override
    public void recharge() {
        if (connector) {
            System.out.println("Recharge started");
            System.out.println("Recharge finished");
        } else {
            throw new IllegalStateException("Connect MicroUsb first");
        }
    }
}

interface MicroUsbPhone {
    void recharge();
    void useMicroUsb();
}

interface LightningPhone {
    void recharge();
    void useLightning();
}

class iPhone implements LightningPhone {
    private boolean connector;

    @Override
    public void useLightning() {
        connector = true;
        System.out.println("Lightning connected");
    }

    @Override
    public void recharge() {
        if (connector) {
            System.out.println("Recharge started");
            System.out.println("Recharge finished");
        } else {
            throw new IllegalStateException("Connect Lightning first");
        }
    }
}

class LightningPhoneAsMicroUsbPhone implements MicroUsbPhone {
    private final LightningPhone lightningPhone;

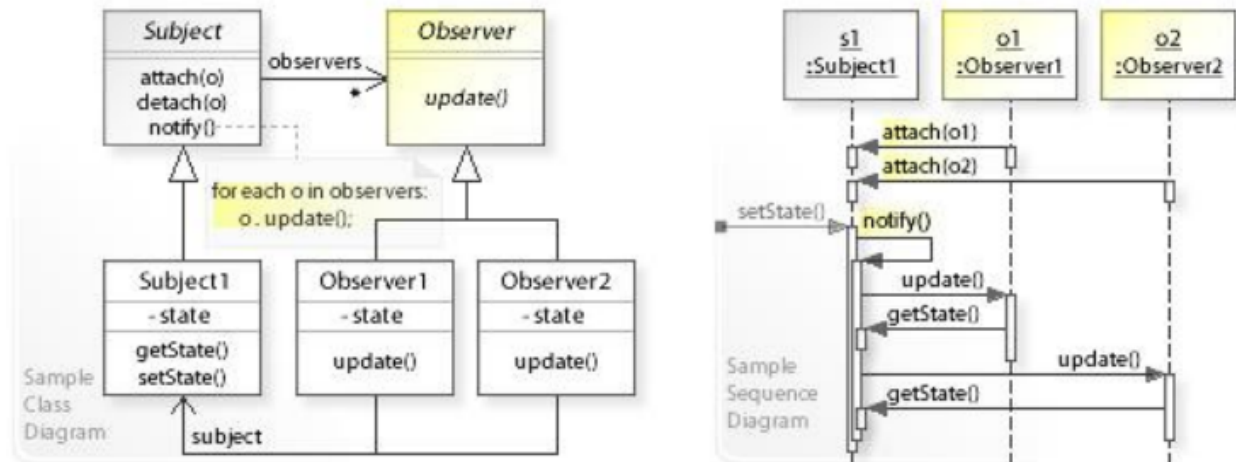
    public LightningPhoneAsMicroUsbPhone(LightningPhone lightningPhone) {
        this.lightningPhone = lightningPhone;
    }

    @Override
    public void useMicroUsb() {
        System.out.println("MicroUsb connected");
        lightningPhone.useLightning();
    }

    @Override
    public void recharge() {
        lightningPhone.recharge();
    }
}

```

Observer Pattern



- AKA dependents, publish-subscribe
- **intent:** define a one-to-many dependency between objects so that when one object changes state, all its dependent are notified and updated automatically
- Use the observer pattern when:
 - when an abstraction has two aspects, one dependent on the other. Encapsulating these aspects in separate objects lets you vary and reuse them independently
 - when a change to one object requires changing others, and you don't know how many objects need to be changed
 - when an object should be able to notify other objects without making assumptions about who these objects are. In other words, you don't want these objects tightly coupled.

Singleton Pattern



- **Intent:** ensure a class only has one instance, and provide a global point of access to it
- Use the singleton pattern when:
 - there must be exactly one instance of a class, and it must be accessible to clients from a well-known access point
 - when the sole instance should be extensible by subclassing, and clients should be able to use an extended instance without modifying their code

- Example:

```
public final class Singleton {  
  
    private static final Singleton INSTANCE = new Singleton();  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return INSTANCE;  
    }  
}
```

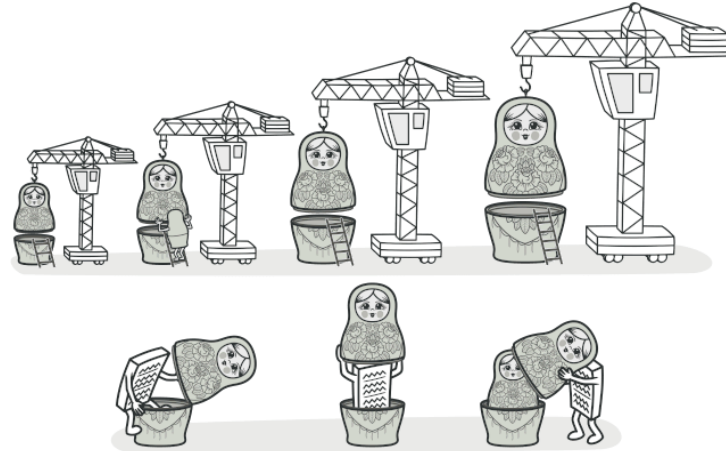
- Problems with singleton:

- they are generally used as a **global instance**, which hides the dependencies of your application in your code instead of exposing them through the interfaces. Making something global to avoid passing it around is a code smell.
- they violate the single responsibility principle by controlling their own creation and lifecycle
- they inherently cause code to be tightly coupled. this makes faking them out under test rather difficult in many cases
- they carry state around for the lifetime of the application. another hit to testing since you can end up with a situation where tests need to be ordered which is a big no for unit tests (each unit test should be independent from the other)

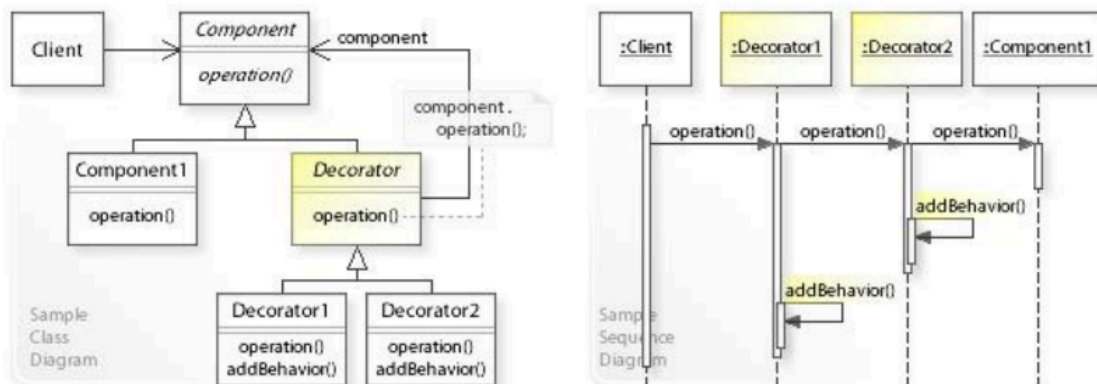
- Alternative: **Dependency injection**

```
// Contoh dengan singleton  
public final class SingletonService { ... }  
public class Client {  
    public String greet() {  
        return "Hello " + SingletonService.getInstance().getName();  
    }  
}  
...  
new Client().greet();  
  
// Contoh dengan dependency injection  
public class Client {  
    private Service service;  
  
    Client(Service service) { this.service = service; }  
  
    public String greet() { return "Hello " + service.getName(); }  
}  
...  
new Client(new SomeService()).greet();
```

Decorator Pattern



- Problems solved:
 - responsibilities should be added to (and removed from) an object dynamically at runtime
 - a flexible alternative to subclassing for extending functionality should be provided
 - when using subclassing, different subclasses extend a class in different ways. But an extension is bound to the class at compile-time and can't be changed at runtime
- The decorator pattern define decorator objects that:



- implement the interface of the extended (decorated) object (component) transparently by forwarding all requests to it
- perform additional functionality before/after forwarding a request

This allows working with different decorator objects to extend the functionality of an object dynamically at runtime

- Example:

```
// The interface Coffee defines the functionality of Coffee implemented by
// decorator
public interface Coffee {
    public double cost(); // Returns the cost of the coffee
    public String ingredients(); // Returns the ingredients of the coffee
}

// Extension of a simple coffee without any extra ingredients
public class SimpleCoffee implements Coffee {
    @Override
    public double cost() {
        return 1;
    }

    @Override
    public String ingredients() {
        return "Coffee";
    }
}

// Abstract decorator class - note that it implements Coffee interface
public abstract class CoffeeDecorator implements Coffee {
    private final Coffee decoratedCoffee;

    public CoffeeDecorator(Coffee c) {
        this.decoratedCoffee = c;
    }

    @Override
    public double cost() { // Implementing methods of the interface
        return decoratedCoffee.cost();
    }

    @Override
    public String ingredients() {
        return decoratedCoffee.ingredients();
    }
}
```



```

// Decorator WithMilk mixes milk into coffee.
// Note it extends CoffeeDecorator.
class WithMilk extends CoffeeDecorator {
    public WithMilk(Coffee c) { super(c); }

    @Override
    public double cost() { return super.cost() + 0.5; }

    @Override
    public String ingredients() { return super.ingredients() + ", Milk"; }
}

// Decorator WithSprinkles mixes sprinkles onto coffee.
// Note it extends CoffeeDecorator.
class WithSprinkles extends CoffeeDecorator {
    public WithSprinkles(Coffee c) { super(c); }

    @Override
    public double cost() { return super.cost() + 0.2; }

    @Override
    public String ingredients() {
        return super.ingredients() + ", Sprinkles";
    }
}

public class Main {
    public static void printInfo(Coffee c) {
        System.out.println("Cost: $" + c.cost() +
            "; Ingredients: " + c.ingredients());
    }

    public static void main(String[] args) {
        Coffee c = new SimpleCoffee();
        printInfo(c);

        c = new WithMilk(c);
        printInfo(c);

        c = new WithSprinkles(c);
        printInfo(c);
    }
}

```

Output:

```

Cost: $1.0; Ingredients: Coffee
Cost: $1.5; Ingredients: Coffee, Milk
Cost: $1.7; Ingredients: Coffee, Milk, Sprinkles

```

